



Auditoría Técnica y Funcional Completa

Diagnóstico objetivo del estado actual del sistema — arquitectura, seguridad, funcionalidad y preparación para producción.

Fecha	Versión App	Fases	Estado
Junio 2026	Next.js 16.2.7	10 fases	Diagnóstico

PUNTAJES FINALES

Arquitectura:	7/10	Backend:	6/10	Frontend:	5/10
Seguridad:	3/10	Escalabilidad:	6/10	Mantenibilidad:	5/10

1.1 Stack Tecnológico

Framework	Next.js 16.2.7 — App Router, Server Components, force-dynamic
ORM	Prisma 7.8.0 con @prisma/adapters-pg (PostgreSQL directo)
Base de datos	PostgreSQL en Railway (producción) / local para desarrollo
Autenticación	NextAuth.js 4.x — CredentialsProvider, JWT 8h
UI	Tailwind CSS 4.x, Lucide React, jsPDF 4.2.1
Runtime	Node.js — deploy en Railway (plataforma PaaS)
Lenguaje	TypeScript 5.x — build limpio, 0 errores

1.2 Módulos y Rutas de la Aplicación

Ruta	Descripción	Rol mínimo
/dashboard	Panel de métricas generales	Admin
/empresas	CRUD de empresas clientes	Admin, Contador
/accesos	Gestión de credenciales externas	Admin, Contador
/calendario	Calendario tributario con fechas	Admin, Contador
/nomina	Lista de empresas con nómina	Todos
/nomina/[id]	Gestión de reportes y novedades por empresa	Todos
/usuarios	Administración de usuarios del sistema	Admin
/configuracion	Perfil y configuración de cuenta	Admin
/login	Página de autenticación	Público

1.3 APIs REST — Endpoints Identificados

Método	Ruta	Descripción	Auth
POST	/api/auth/[...nextauth]	Autenticación NextAuth	'
GET/POST	/api/app-users	Usuarios del sistema	'
PATCH/DELETE	/api/app-users/[id]	Modificar/eliminar usuario	'
GET/POST	/api/empresas	CRUD empresas	'
GET/PATCH/DELETE	/api/empresas/[id]	Empresa individual	'
GET/POST	/api/accesos	Credenciales externas	' sin auth
GET/PATCH/DELETE	/api/accesos/[id]	Acceso individual	' sin auth
GET/POST	/api/calendario	Eventos tributarios	' sin auth
GET/PUT/DELETE	/api/calendario/[id]	Evento individual	' sin auth
GET/POST	/api/nomina/empresas	Empresas con nómina	'
GET/PATCH	/api/nomina/empresas/[id]	Empresa nómina individual	'
GET/POST	/api/nomina/reportes	Reportes de nómina	'
GET/PATCH	/api/nomina/reportes/[id]	Reporte individual	'

GET	/api/nomina/reportes/[id]/export	Exportar CSV/texto	'
GET/POST	/api/nomina/empleados	Empleados	'
GET/PATCH/DELETE	/api/nomina/empleados/[id]	Empleado individual	'
GET/POST	/api/nomina/novedades	Novedades de nómina	'
GET/PATCH/DELETE	/api/nomina/novedades/[id]	Novedad individual	'
GET/POST	/api/nomina/libranzas	Libranzas / Préstamos	'
GET/PATCH/DELETE	/api/nomina/libranzas/[id]	Libranza individual	'
GET	/api/nomina/auditorias/reportes/[id]	Auditoría de reporte	'
GET/POST	/api/dashboard	Métricas del dashboard	' sin auth
POST	/api/seed	Seed de datos iniciales	' sin auth

1.4 Modelos Prisma

User	Usuarios del sistema (NextAuth)	id, email, name, role, isActive, empresalds
Empresa	Empresas clientes	id, nit, razonSocial, tipoPersona, regimen, activo
Acceso	Credenciales externas cifradas	id, empresald, tipo, plataforma, contrasena
CalendarioTributario	Eventos tributarios	id, titulo, fecha, tipo, empresald?
EmpresaNomina	Conf. nómina por empresa	id, empresald, modo (SIMPLE/COMPLETO)
EmpleadoNomina	Empleados de nómina	id, empresald, nombre, cargo, salarioBase
ReporteNomina	Reportes mensuales	id, empresald, mes, año, estado, totalNomina
NovedadNomina	Novedades por reporte	id, reporteld, empleadold, tipoNovedad, valor
Libranza	Libranzas y préstamos	id, empresald, empleadold, tipo, cuotaActual
AuditoriaReporte	Historial de cambios de estado	id, reporteld, accion, realizadoPorId

1.5 Enumeraciones (Prisma Enums)

TipoNovedad	18 valores	HORAS_EXTRAS, INCAPACIDAD, VACACIONES, PRIMA_SERVICIOS, CESANTIAS, INTERESES_CESANTIAS, LICENCIA_MATERNIDAD, LICENCIA_PATERNIDAD,
EstadoReporteNomina	6 valores	BORRADOR, ENVIADA, REVISADA, APROBADA, REABIERTA, CORREGIDA
ModoFlujoNomina	2 valores	SIMPLE, COMPLETO
UserRole	4 valores	ADMIN, ANALYST, NOMINA, CLIENT
TipoAcceso	Múltiple	DIAN, SEGURIDAD_SOCIAL, BANCO, SOFTWARE_CONTABLE, OTROS
TipoPersona	2 valores	NATURAL, JURIDICA
RegimenTributario	4 valores	SIMPLE, COMUN, ESPECIAL, GRAN_CONTRIBUYENTE

2.1 Módulo de Empresas

- CRUD completo (crear, listar, editar, eliminar con confirmación)
- Filtro por NIT / razón social funcional
- Soft delete con campo 'activo'
- Página de detalle (/empresas/[id]) usa datos mock hardcodeados — no conectada a BD
- & Sin paginación server-side (carga todos los registros en memoria)
- & Sin validación de NIT único en el frontend

2.2 Módulo de Accesos

- CRUD completo de credenciales externas
- Contraseña enmascarada en la lista (•••••)
- Botón 'revelar contraseña' con llamada GET /api/accesos/[id]
- Cifrado de contraseña: XOR con clave fija — trivialmente reversible (no es cifrado real)
- Sin autenticación en los endpoints GET/POST/PATCH/DELETE de /api/accesos
- Sin control de acceso por empresa (cualquier usuario ve todos los accesos)

2.3 Módulo de Calendario Tributario

- Vista de calendario con eventos por mes funcional visualmente
- Formulario de creación de eventos presente
- API /api/calendario sin autenticación
- & Los datos mostrados en la UI provienen mayormente de mocks locales
- & Sin notificaciones ni recordatorios automáticos

2.4 Módulo de Nómina — Reportes

- Creación de reportes por empresa, mes y año
- Flujo de estados completo: BORRADOR ! ENVIADA ! REVISADA ! APROBADA
- Historial de auditoría por reporte (AuditoriaReporte)
- Exportación a CSV / texto con formato estructurado
- Generación automática de cuotas libranza/préstamo al crear reportes
- Sin cálculo automático de salario neto / total nómina
- totalNomina en BD siempre es 0 (no se calcula)

2.5 Módulo de Nómina — Novedades (18 tipos)

- HORAS_EXTRAS: valor, horas, tipo (diurnas/nocturnas/festivas)
- INCAPACIDAD: días, tipo (enfermedad/accidente/licencia)
- VACACIONES: días, fecha inicio/fin
- PRIMA_SERVICIOS, CESANTIAS, INTERESES_CESANTIAS: valor simple
- LICENCIAS (maternidad, paternidad, remunerada, no remunerada): días
- SUSPENSION_DISCIPLINARIA: días
- AUSENTISMO: días y descripción
- LIBRANZA: vínculo a Libranza model, cuotas automáticas, entidad
- PRESTAMO (nuevo): misma lógica que Libranza con tipo='PRESTAMO'
- DESCUENTO_AUTORIZADO (nuevo): valor + observación obligatoria
- BONIFICACION: valor y descripción
- DEDUCCION_OTRO / NOVEDAD_OTRO: valor y descripción genérica
- & Sin validación de límites legales (ej. máx días vacaciones, horas extras legales)

2.6 Módulo de Usuarios

- CRUD de usuarios con roles (ADMIN, ANALYST, NOMINA, CLIENT)
- Asignación de empresas a usuarios tipo CLIENT

- Eliminación con fallback a soft-delete cuando hay FK activas
- Filtro por rol y estado (activo/inactivo)
- & Sin recuperación de contraseña / reset por email
- Contraseñas hasheadas con bcrypt (correcto)

2.7 Dashboard

- Tarjetas de métricas: empresas activas, reportes del mes, alertas
- Gráfica de distribución de novedades visible
- API /api/dashboard sin autenticación
- & Datos de gráficas pueden ser parcialmente mock

3.1 Roles Definidos en el Sistema

ADMIN	Acceso total — todas las rutas y funciones del sistema
ANALYST (Contador)	Gestión de empresas, accesos, calendario, nómina — sin admin de usuarios
NOMINA	Acceso específico a módulo de nómina — rol intermedio
CLIENT (Cliente)	Acceso solo a /nomina de sus empresas asignadas

3.2 Implementación de Guards

- `src/proxy.ts` (naming crítico): el archivo no se llama `middleware.ts` — puede no ejecutarse como middleware de Next.js
- Las rutas de nómina usan `getNominaSession()` + `canAccess()` + `isContador()` — correctamente implementado
- Las rutas de API de empresas y usuarios validan sesión con `getServerSession()`
- `/api/accesos`, `/api/calendario`, `/api/dashboard`: sin ninguna validación de sesión
- Un cliente puede enviar peticiones directas a `/api/accesos` y ver credenciales de otras empresas
- & El rol NOMINA existe en la BD pero no tiene rutas ni UI específica — sin uso real

3.3 Mapeo Sidebar vs Permisos Reales

El sidebar muestra rutas según el rol del usuario. Sin embargo, si `proxy.ts` no corre como middleware, las páginas son técnicamente accesibles sin autenticación desde el navegador. La protección efectiva depende únicamente de la verificación a nivel de API.

- ADMIN: ve Dashboard, Empresas, Accesos, Calendario, Nómina, Configuración, Usuarios
- ANALYST/Contador: ve Empresas, Accesos, Calendario, Nómina
- CLIENT: ve solo Nómina (redirigido a `/nomina` al login)
- & No existe página de 'acceso denegado' — los redirects son al login

4.1 Hallazgos ALTO Riesgo

CRÍTICO — 12 endpoints sin autenticación

Los siguientes endpoints no verifican sesión y son accesibles por cualquier cliente HTTP sin credenciales:

- ' /api/accesos (GET, POST)
- ' /api/accesos/[id] (GET, PATCH, DELETE)
- ' /api/calendario (GET, POST)
- ' /api/calendario/[id] (GET, PUT, DELETE)
- ' /api/dashboard (GET, POST)
- ' /api/seed (POST — puede resetear datos)

CRÍTICO — Cifrado de contraseñas XOR (trivialmente reversible)

El módulo /api/accesos usa XOR byte-a-byte con una clave fija (ENCRYPTION_KEY) para 'cifrar' contraseñas de terceros. Cualquier atacante con acceso a la BD o a ENCRYPTION_KEY puede obtener todas las contraseñas en texto plano con una sola operación. Esto no es cifrado — es ofuscación. Se requiere AES-256-GCM o similar.

CRÍTICO — proxy.ts no es reconocido como middleware por Next.js

Next.js espera un archivo llamado exactamente middleware.ts (o middleware.js) en la raíz del proyecto o en src/. El archivo src/proxy.ts no se ejecuta automáticamente — la protección de rutas de página puede estar completamente inactiva.

4.2 Hallazgos MEDIO Riesgo

- & Sistema de autenticación legado (user-store.ts) con contraseñas en texto plano potencialmente persistidas
- & JWT sin rotación — tokens de 8 horas sin blacklist al logout
- & Sin rate limiting en /api/auth/[...nextauth] (vulnerable a fuerza bruta)
- & Sin CSRF protection explícita en formularios de mutación
- & ENCRYPTION_KEY tiene valor fallback 'default-key' si no está en .env (riesgo en prod)

4.3 Hallazgos BAJO Riesgo

- ! Logs de error en consola pueden exponer stack traces en producción
- ! Sin Content Security Policy (CSP) headers configurados
- ! AuditoriaNovedad tiene onDelete:Cascade — eliminar una novedad borra el audit trail
- ! Sin validación de tamaño/tipo en inputs de texto libre (observaciones, etc.)

5.1 Estructura General

- 10 modelos Prisma bien relacionados con FKs declaradas
- Uso de UUIDs (cuid()) como PK — correcto para distribución
- Timestamps automáticos (createdAt, updatedAt) en todos los modelos
- Soft delete implementado en Empresa y User (campo 'activo'/'isActive')
- & NovedadNomina no tiene soft delete — las novedades se eliminan físicamente

5.2 Índices y Performance

- No hay índices explícitos declarados en schema.prisma (más allá de los PK automáticos)
- & Consultas frecuentes sin índice: ReporteNomina(empresald, mes, año), NovedadNomina(reporteld), Libranza(empleadold)
- & findMany sin paginación en varias APIs — riesgo de timeouts con datos reales
- & Libranza.tipo es String sin restricción de enum — posibles valores inválidos

5.3 Relaciones y Constraints

- onDelete: Cascade en AuditoriaNovedad! eliminar novedad destruye el historial
- & User.empresalds es String[] (array de strings) — no hay FK real a Empresa.id
- & CalendarioTributario.empresald es opcional — eventos globales y por empresa mezclados sin distinción clara
- & EmpresaNomina y Empresa son entidades separadas — posible desincronización

5.4 Migraciones

- Sistema de migraciones Prisma activo y funcional
- Migración add_prestamo_descuento_autorizado aplicada exitosamente
- & No hay migraciones de seed de datos de prueba controladas por versión
- /api/seed expuesto sin protección — riesgo de reset accidental en producción

5.5 Calidad de Datos

- totalNomina en ReporteNomina siempre almacena 0 — no hay lógica de cálculo
- Libranza.valorCuota es Decimal — correcto para valores monetarios
- & NovedadNomina.valor puede ser null — algunas novedades requieren valor pero no hay constraint BD
- & Fechas almacenadas como DateTime (con zona horaria UTC) — puede causar problemas de visualización en Colombia (UTC-5)

6.1 Flujo de Estados del Reporte

El sistema implementa un flujo de aprobación multi-etapa para los reportes de nómina:

BORRADOR	Estado inicial — editable por contador/admin
ENVIADA	Enviada al cliente para revisión — solo lectura para contador
REVISADA	Cliente confirma revisión — esperando aprobación
APROBADA	Aprobación final — reporte cerrado
REABIERTA	Admin reabre el reporte para correcciones
CORREGIDA	Corrección aplicada — vuelve al flujo

6.2 Modos de Flujo (ModoFlujoNomina)

- SIMPLE: cliente puede aprobar directamente sin pasar por REVISADA
- COMPLETO: flujo completo BORRADOR ! ENVIADA ! REVISADA ! APROBADA
- Configurado por empresa en EmpresaNomina.modoflujo
- & El cliente no puede ver qué modo está activo para su empresa

6.3 Libranzas y Préstamos

- Modelo Libranza con cuotaActual, numeroCuotas, valorCuota, entidad
- Generación automática de cuotas en períodos subsiguientes (generar-nominas.ts)
- PRESTAMO usa mismo modelo Libranza con campo tipo='PRESTAMO'
- Prevención de eliminar libranza con cuotas pagadas (solo desactivar)
- Prevención de modificar valorCuota/numeroCuotas con cuotas ya pagadas
- & tipo es String — debería ser enum para prevenir valores inválidos

6.4 Cálculos de Nómina — Estado Actual

HALLAZGO CRÍTICO: El sistema gestiona el registro de novedades pero NO calcula el salario neto de los empleados:

- No hay fórmula de salario base + bonificaciones - deducciones implementada
- ReporteNomina.totalNomina = 0 siempre (campo existe pero no se calcula)
- No hay cálculo de parafiscales, aportes a seguridad social, retención en la fuente
- & La exportación incluye novedades pero no el total neto por empleado
- & El sistema funciona actualmente como un REGISTRO de novedades, no como una nómina real

6.5 Auditoría de Novedades

- AuditoriaReporte registra cambios de estado con usuario y timestamp
- & No hay auditoría de creación/edición de novedades individuales
- AuditoriaNovedad model existe pero onDelete:Cascade destruye registros al borrar novedad

7.1 Login y Autenticación

- Pantalla de login profesional con degradado, logo y validación visual
- Spinner de carga durante autenticación, botón desactivado
- Redirección por rol (`cliente ! /nomina`, `contador ! /empresas`, `admin ! /dashboard`)
- & 'Recordarme' y '¿Olvidaste tu contraseña?' no tienen funcionalidad implementada
- Sin página de recuperación de contraseña

7.2 Sidebar y Navegación

- Sidebar colapsable en desktop con iconos, estados activos correctos
- Navegación móvil con bottom nav (primeros 5 ítems)
- Avatar con iniciales del usuario, email visible
- Logout funcional
- & Sin breadcrumbs en páginas de detalle (`empresas/[id]`, `nomina/[id]`)

7.3 Módulo de Nómina — Formulario de Novedades

- Selector de tipo de novedad con 18 opciones correctamente renderizado
- Campos dinámicos por tipo de novedad (`renderFields()`)
- Creación de libranza inline con todos los campos necesarios
- Chips de resumen de novedades por empleado en el reporte
- Modo de edición de novedades existentes funcional
- & Sin confirmación visual al guardar una novedad (solo desaparece el form)
- & Sin feedback de error amigable cuando falla la API

7.4 Tabla de Reportes

- Lista de reportes con estado visual (badges de color por estado)
- Botones de cambio de estado contextuales según rol
- Historial de auditoría visible por reporte
- & Sin paginación — todos los reportes de todos los meses visibles a la vez
- & Sin filtro por año o estado en la lista de reportes

7.5 Responsividad y Accesibilidad

- Diseño responsive con breakpoints md: para desktop/móvil
- Bottom nav para móvil — buena experiencia básica
- Inputs con labels correctamente asociados (`htmlFor`)
- & Sin atributos ARIA en componentes custom (dropdowns, modales)
- & Contraste de colores no verificado contra WCAG 2.1
- & Sin manejo de estados vacíos consistente en todas las tablas

8.1 Infraestructura (Railway)

- Deploy en Railway con PostgreSQL administrado — buena elección para startup
- Variables de entorno necesarias: DATABASE_URL, NEXTAUTH_SECRET, NEXTAUTH_URL, ENCRYPTION_KEY
- & Sin configuración de backups automáticos documentada
- & Sin configuración de alertas de uptime / error rate
- & Sin CDN para assets estáticos

8.2 Variables de Entorno y Configuración

- NEXTAUTH_SECRET: correctamente externalizado
- DATABASE_URL: correctamente externalizado
- ENCRYPTION_KEY: externalizado pero con fallback inseguro 'default-key'
- & Sin validación de variables de entorno al arrancar (ej. con zod/invalid)
- & Sin archivo .env.example o documentación de variables requeridas

8.3 Logging y Monitoreo

- & console.error() en catch blocks — suficiente para desarrollo, insuficiente en prod
- & Sin integración con Sentry, Datadog u otra plataforma de observabilidad
- & Sin métricas de performance de queries Prisma
- & Sin health check endpoint (/api/health)

8.4 Build y CI/CD

- Build de Next.js pasa limpio — 0 errores TypeScript
- & Sin pipeline de CI/CD automatizado (GitHub Actions u otro)
- & Sin proceso de PR review / code review documentado
- & Sin ambiente de staging separado de producción

8.5 Datos y Privacidad

- & Datos de nómina son datos sensibles — sin política de retención documentada
- Sin encriptación en reposo de datos sensibles más allá del cifrado XOR de accesos
- & Sin proceso de eliminación de datos de cliente al terminar contrato
- & Sin términos de servicio o política de privacidad en la plataforma

9.1 Cobertura de Tests

0 tests

No existe ningún archivo de tests en el proyecto.

No se encontraron archivos *.test.ts, *.spec.ts, ni directorios __tests__ en ningún módulo del proyecto. No hay framework de testing configurado (Jest, Vitest, Playwright, Cypress).

9.2 Áreas Críticas Sin Cobertura

- Lógica de autenticación y autorización (roles, canAccess)
- Flujo de estados de reportes (transiciones válidas e inválidas)
- Validaciones de novedades de nómina (18 tipos con reglas distintas)
- Lógica de cuotas de libranza/préstamo y avance automático
- APIs sin autenticación (verificar que ahora sí protejan)
- Cifrado/descifrado de contraseñas de accesos

9.3 Lo que Sí Funciona como Control de Calidad

- TypeScript estricto — errores de tipo detectados en compile-time
- Build de Next.js como integración básica — verifica importaciones y syntax
- Prisma type safety — errores de schema detectados en generate

9.4 Recomendación de Testing

Para un sistema de nómina que maneja datos financieros sensibles, se recomienda mínimamente:

- Vitest para unit tests de lógica de negocio (validaciones, cálculos, estados)
- Playwright para E2E del flujo crítico: login !' crear reporte !' agregar novedad !' ap
- Supertest para integration tests de APIs con DB de test aislada

10.1 Tabla de Puntuaciones por Dimensión

Arquitectura		7/10	Stack moderno y bien elegido. App Router de Next.js correctamente usado. Separación de concerns aceptable.
Backend / APIs		6/10	APIs de nómina bien estructuradas. 12 endpoints sin autenticación, lo que es bloqueante para producción.
Frontend / UX		5/10	UI profesional y funcional. Faltan estados vacíos, confirmaciones y feedback de errores.
Seguridad		3/10	Múltiples vulnerabilidades críticas: endpoints abiertos, XOR 'cifrado', middleware inactivo.
Escalabilidad		6/10	Railway + Prisma escala razonablemente. Sin índices explícitos ni paginación server-side.
Mantenibilidad		5/10	0 tests, código parcialmente duplicado (auth legacy), sin CI/CD.
Funcionalidad		6/10	Registro de novedades completo. Sin cálculo de nómina real — el valor más crítico falta.
Preparación Prod		4/10	No listo para clientes reales sin resolver seguridad, tests, cálculo de nómina.

10.2 Resumen Ejecutivo

Estado General del Sistema

El sistema tiene una arquitectura sólida y una buena base de código. El módulo de nómina es el más completo y funcional del sistema, con 18 tipos de novedades, flujo de aprobación multi-etapa y generación automática de cuotas.

Sin embargo, hay 3 bloqueantes CRÍTICOS antes de poder usarlo con clientes reales:

1. Seguridad: 12 APIs sin autenticación + XOR no es cifrado real + middleware posiblemente inactivo.
2. Funcionalidad: El sistema no calcula nómina — solo registra novedades. totalNomina = 0 siempre.
3. Calidad: 0 tests en un sistema financiero es un riesgo operacional inaceptable.

10.3 Roadmap de Correcciones Prioritarias

P0 — Bloqueante

- ! Agregar autenticación a /api/accesos, /api/calendario, /api/dashboard, /api/seed
- ! Renombrar proxy.ts ! middleware.ts y verificar que el guard funciona
- ! Reemplazar XOR con AES-256-GCM para encriptación de contraseñas

P1 — Crítico

- !' Implementar cálculo de nómina: salario base ± novedades = total neto
- !' Agregar índices en BD para queries frecuentes
- !' Configurar tests básicos (Vitest) para lógica de nómina y autenticación

P2 — Importante

- !' Completar página de detalle de empresa con datos reales
- !' Agregar paginación server-side en listas con muchos registros
- !' Eliminar sistema de autenticación legado (user-store.ts)
- !' Agregar health check endpoint y logging estructurado

P3 — Mejora

- !' Implementar recuperación de contraseña por email
- !' Agregar filtros de año y estado en lista de reportes de nómina
- !' Configurar CI/CD con GitHub Actions
- !' Validar variables de entorno al arrancar con zod